Abdulla Murshid KV
Associate Technical Consultant

QA CAPSTONE PROJECT

Playwright Test Automation Framework

End-to-end QA automation for **automationexercise.com** — covering the full user journey across 3 browsers.

FRAMEWORK	LANGUAGE	TEST CASES	BROWSERS	GITHUB
Playwright v1.64.1	TypeScript	11 Tests	Chromium · Firefox · WebKit	github.com/Murshid98/playwright-qa-capstone

PROJECT OVERVIEW

What was built

TEST CASES	BROWSERS	PAGE OBJECTS	PASS RATE
11	3	7	100%
Across 6 test types	Chrome · Firefox · Safari	Full POM design pattern	All tests passing

A production-grade framework testing registration, login, cart, checkout, search, accessibility, visual regression, and API mocking on a live e-commerce site behind Cloudflare.

TECHNOLOGY

Tech Stack

TEST FRAMEWORK

Playwright

Browser automation, built-in waits, cross-browser

LANGUAGE

TypeScript

Type-safe test authoring, compile-time checks

ACCESSIBILITY

axe-core

WCAG 2.1 AA automated scanning

FAKE DATA

Faker.js

Unique emails and names per run

CI / CD

GitHub Actions

Auto-run on push, HTML + JUnit reports

REPORTS

HTML + JUnit

Visual report + CI-dashboard XML

DESIGN PATTERN

Framework Architecture

TEST SPECS	11 `.spec.ts` files — call page object methods and assert results. No raw Playwright code in tests.
FIXTURES	`fixtures/pageFixture.ts` — injects all 7 page objects per test + blocks 7 ad networks globally via `page.route()`.
PAGE OBJECTS	7 classes in `pages/` — each maps one website page. All extend `BasePage` for shared retry & navigation logic.
UTILITIES	`utils/` — randomDataGenerator, authHelper, waitHelper, excelReader — shared helpers across all tests.
TEST DATA	`testdata/` — static JSON (payment cards, addresses). Dynamic data generated fresh each run with Faker.js.

TEST COVERAGE

11 Test Cases

TC01
User Registration
Auth

TC02
Login Valid Credentials
Auth

TC03
Logout Clears Session
Auth

TC04
Product Search
Functional

TC05
Add Product to Cart
Functional

TC06
Remove from Cart
Functional

TC07
End-to-End Checkout
E2E

TC08
Contact Us Form
Functional

TC09
WCAG 2.1 Accessibility
A11y

TC10
Visual Regression
Visual

TC11
Network Mock / API
Network

PROBLEM SOLVING

Key Challenges Solved

Cloudflare 524 Timeouts
Site returns 524 errors as valid HTML. Added detection for `"timeout occurred"` after every navigation with automatic page reload and retry logic.

Ad Overlays Blocking Clicks
Full-page iFOREX modals covered the checkout button. Blocked 7 ad networks via `page.route()` globally in fixtures + `dismissAdModal()` as fallback.

Visual Test Font Timeout (WebKit)
WebKit's `document.fonts.ready` exceeded 5s. Fixed by blocking font file requests so fonts resolve instantly, plus 60s snapshot timeout.

Navigation Race Condition
`waitForLoadState` resolved on the current page before navigation started. Replaced with `clickAndWaitForNav()` using URL-based detection.

AUTOMATION

CI / CD Pipeline

TRIGGER

✓ Every push to **main** branch

✓ Every pull request

✓ Manual run with browser selection

ENVIRONMENT

Ubuntu Latest (GitHub hosted)

Node.js LTS & npm ci

Chromium + OS dependencies

TEST EXECUTION

TC01–TC09, TC11 on Chromium

TC10 runs with `--update-snapshots`

retries: 2 on failure in CI

ARTIFACTS UPLOADED

📊 HTML Report → retained 30 days

📋 JUnit XML → retained 30 days

🔍 Traces (on failure) → 7 days

RESULTS

All Tests Passing

Chromium	TC01 ✓	TC02 ✓	TC03 ✓	TC04 ✓	TC05 ✓	TC06 ✓	TC07 ✓	TC08 ✓	TC09 ✓	TC10 ✓	TC11 ✓
Firefox	TC01 ✓	TC02 ✓	TC03 ✓	TC04 ✓	TC05 ✓	TC06 ✓	TC07 ✓	TC08 ✓	TC09 ✓	TC10 ✓	TC11 ✓
WebKit (Safari)	TC01 ✓	TC02 ✓	TC03 ✓	TC04 ✓	TC05 ✓	TC06 ✓	TC07 ✓	TC08 ✓	TC09 ✓	TC10 ✓	TC11 ✓

33 total test executions across 3 browsers — **33 / 33 passed**.

REFLECTION

What I Learned

TECHNICAL SKILLS

- Playwright fixtures & Page Object Model
- TypeScript for type-safe test authoring
- Visual regression with per-OS baselines
- Accessibility testing with axe-core
- API mocking with `page.route()`

REAL-WORLD QA

- Handling flaky tests on a live site
- Ad overlay detection & mitigation
- Cloudflare timeout recovery strategy
- CI/CD pipeline setup & reporting
- Cross-browser compatibility testing

WHAT I'D IMPROVE

- Parallel workers (needs mock server)
- API-based test user setup & teardown
- Negative test cases for invalid inputs
- Multi-environment config (staging/prod)

KEY TAKEAWAY

- Testing a live site requires resilience
- POM keeps tests maintainable at scale
- Fixtures eliminate test setup boilerplate
- CI catches regressions before they ship

THANK YOU

Questions?

Built with **Playwright + TypeScript** for automationexercise.com — 11 tests, 3 browsers, 100% pass rate.

PRESENTER
Abdulla Murshid KV

DESIGNATION
Associate Technical Consultant

REPOSITORY
github.com/Murshid98/playwright-qa-capstone

RUN COMMAND
```
npx playwright test --headed
```